

¿Qué es REST y para que sirve?

REST (por sus siglas en inglés, Representational State Transfer o Transferencia de Estado Representacional) es un estilo de arquitectura de software que define un conjunto de reglas para la comunicación entre sistemas distribuidos a través del protocolo HTTP.

Sirve para estandarizar cómo las aplicaciones intercambian información. Al obligar a que los sistemas compartan representaciones del estado de sus recursos de forma predecible, permite crear arquitecturas altamente escalables, rápidas y tolerantes a fallos, aprovechando la infraestructura nativa de Internet.

¿Qué es una API, para que sirve y cual es el objetivo de combinarlo con REST?

Una API (Application Programming Interface) es un contrato de comunicación o conjunto de reglas que permite a diferentes aplicaciones de software "hablar" entre sí. Su propósito es integrar sistemas y exponer funcionalidades abstrayendo por completo el código fuente o la complejidad interna del servidor.

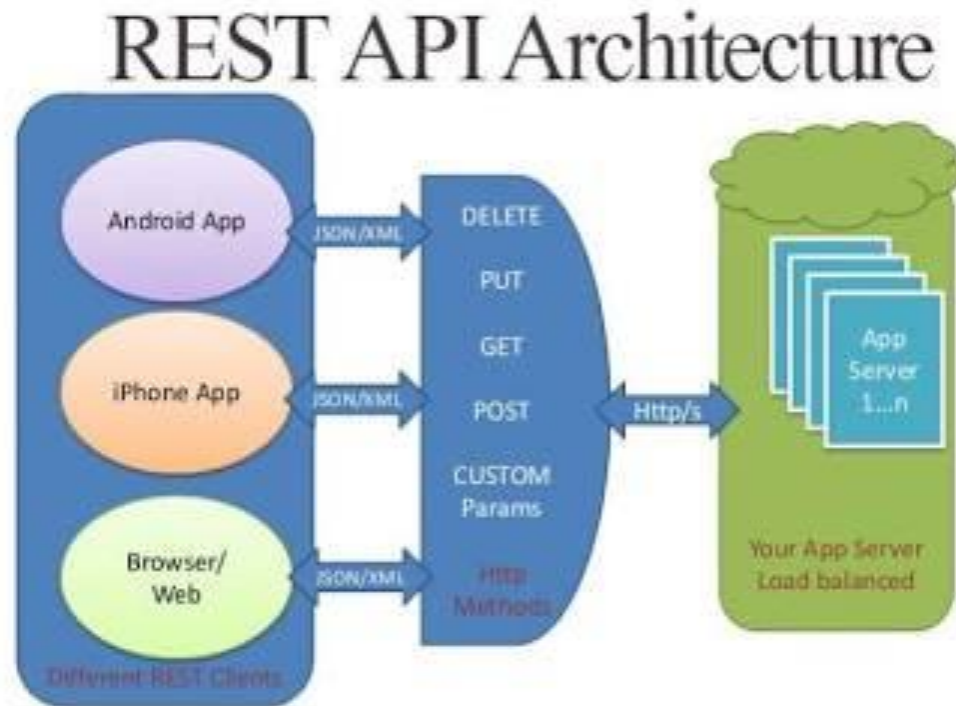
El objetivo de combinarlos (API REST) consiste en utilizar los estándares universales de la web (el protocolo HTTP) para construir servicios ligeros y multiplataforma. Al diseñar una API bajo los principios REST:

- El diseño se orienta a Recursos: Todo gira en torno a sustantivos manejables (ej. "Usuarios", "Facturas", "Productos").
- Se estandariza la Acción: Se utilizan los verbos HTTP nativos (GET, POST, PUT, PATCH, DELETE) para ejecutar operaciones CRUD (Crear, Leer, Actualizar, Borrar).
- Formatos Universales: Los datos viajan en cargas útiles (payloads) ligeras y procesables por cualquier lenguaje, principalmente JSON o XML.

¿Qué es un Endpoint?

Un **Endpoint** (punto de terminación) es la dirección digital exacta (una URL/URI) en la que una API expone un recurso específico para que un cliente interactúe con él.

Realizar un ejemplo Gráfico del Uso de API REST. (Fuente: jonmircha)



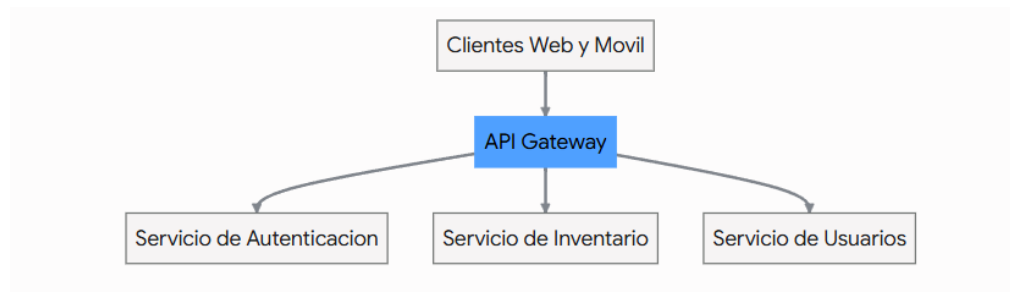
Mencione cuales son los patrones de diseño que se aplican para API. Realizar una explicación de cada uno de ellos.

Los patrones de diseño aplicados en APIs permiten estructurar el código y resolver problemas comunes de comunicación, mantenimiento y rendimiento.

En arquitecturas empresariales, exponer endpoints no es suficiente; la resiliencia y el mantenimiento a escala dependen de la aplicación de patrones de diseño específicos para integraciones web.

- **API Gateway (Fachada)**

En lugar de que una app móvil tenga que hacer peticiones a cinco servicios distintos (usuarios, carrito, inventario, pagos), realiza una sola llamada al API Gateway. Este componente enruta la petición hacia los servicios correspondientes, consolida las respuestas y se encarga de las responsabilidades transversales, como la validación de tokens de autenticación, el balanceo de carga y el *rate limiting* (prevención de abusos de tráfico).



- **Adapter:** Traduce o mapea estructuras de datos externas o heredadas a un formato que la API o el cliente puedan comprender sin errores.
- **Decorator :** Añade funcionalidades adicionales a los objetos o peticiones de forma dinámica (por ejemplo, para registrar logs o modificar cabeceras).
- **Builder :** Facilita la creación de objetos de solicitud o respuesta complejos paso a paso, muy útil para armar payloads JSON limpios.
- **Singleton:** Garantiza que una clase o servicio (como la conexión a una base de datos o un cliente HTTP global) tenga una sola instancia durante el ciclo de vida de la aplicación.
- **Strategy:** Permite intercambiar algoritmos o reglas de negocio (por ejemplo, diferentes métodos de cálculo de impuestos o pasarelas de pago) según los parámetros de la petición.